

- **Consortium network:** In a consortium network, only specific validators are allowed to join the validator network and participate in consensus. However, the network allows any client, transaction processor, or event subscriber to connect to a validator, and accept batches and transactions signed by all transactors.
- **Private network:** In a private network, only specific validators are allowed to join the validator network and participate in a consensus. The validators in the network accept only connections from specific clients. The validators also control whether the client is allowed to submit batches and query specific address prefixes in state. Only specific transaction processors and event subscribers are allowed to connect to the local validator. Batches and transactions that are received by the validator can only be signed by permitted transactors. Transactors can also be restricted to only sending transactions for certain transaction families.

Transactor key permissioning

Transactor key permissioning is performed on the basis of a transactor's public signing key. This includes both batch signers and transaction signers. When the validator receives a batch from a client, it checks both the local configuration and network configuration. Only the batch signers that are permitted in the intersection of the two configurations will be allowed. When the validator receives

a batch from a peer validator, only the network configuration is checked. This behaviour is required to prevent network forks. The following are high level capabilities of transactor key permissioning:

- Allow all batch signers to submit batches, where the validator can be configured to allow all batches to be submitted, regardless of who the batch signer is.
- Allow only specific batch signers to submit batches, where the validator can be configured to only allow certain batch signers to submit batches.
- Allow all transaction signers to submit transactions, where the validator can be configured to allow all transactions to be submitted, regardless of who the transaction signer is.
- Allow only specific transaction signers to submit transactions, where the validator can be configured to only allow certain transaction signers to submit transactions.
- Restrict the type of transactions transactors can sign, where the validator can restrict the transaction signers that are allowed to sign transactions for a specific transaction family.
- Support policy based transactor permissioning, where the validator

can enforce transactor permissions based on defined policies.

Validator key permissioning

Validator key permissioning is performed on the basis of a validator node's public signing key. The following are high level capabilities of validator key permissioning:

- Allow all nodes to join the validator network, whereby the validator network can be configured to allow all validator nodes to join the network. This means that every validator on the network should accept the node as a peer, regardless of its public key.
- Allow only specific nodes to join the validator network, where the validator network can be configured to only allow nodes to join the network if the node's public key is permitted.
- Allow only specific nodes to participate in consensus, where the validator network can be configured to only allow specific nodes to participate in consensus. This is separate from any restrictions enforced by the consensus algorithm. The nodes that are not allowed to participate in consensus can still validate blocks but are not allowed to publish blocks.

In the next post we will take a closer look at Sawtooth events. **END** 

 By: Ashwini Kulkarni

The author is an independent consultant with prior experience in training, who is actively exploring emerging technologies like blockchain and containerisation. She is also certified in cyber law and cyber forensics.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU